In [5]:
```
pip install numpy as np
```

```
Requirement already satisfied: numpy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (2.4.6)
Note: you may need to restart the kernel to use updated packages.
ERROR: Could not find a version that satisfies the requirement as (from versions: no
ne)
ERROR: No matching distribution found for as

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [6]:
```
!pip install pandas as pd
```

```
Requirement already satisfied: pandas in c:\users\hp\desktop\mashrooms .csv\ml_env\l
ib\site-packages (3.0.6)
ERROR: Could not find a version that satisfies the requirement as (from versions: no
ne)
ERROR: No matching distribution found for as

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [7]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("All libraries working!")
```

```
All libraries working!
```

In [8]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

In [9]:
```
from sklearn.preprocessing import LabelEncoder
```

In [10]:
```
import pandas as pd
print(pd.__version__)
```

```
3.0.6
```

In [11]:
```
df=pd.read_csv("mushrooms.csv")
df.head()
```

Out[11]:

	type	cap_shape	cap_surface	cap_color	bruises	odor	gill_attachment	gill_spacing	gi
0	p	x	s	n	t	p	f	c	
1	e	x	s	y	t	a	f	c	
2	e	b	s	w	t	l	f	c	
3	p	x	y	w	t	p	f	c	
4	e	x	s	g	f	n	f	w	

5 rows × 23 columns

In [12]:
```
import pandas as pd
df = pd.read_csv("mushrooms.csv")
```

In [13]:
```
x = df.drop("type", axis=1)
y = df["type"]
```

In [14]:
```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

In [15]:
```
for col in x_train.columns:
    le = LabelEncoder()
    x_train[col] = le.fit_transform(x_train[col])
    x_test[col] = le.transform(x_test[col])

le_y = LabelEncoder()
y_train = le_y.fit_transform(y_train)
y_test = le_y.transform(y_test)
```

In [16]:
```
classifier = DecisionTreeClassifier()
classifier.fit(x_train, y_train)
```

Out[16]:

▾ DecisionTreeClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes

In [17]:
```
prediction = classifier.predict(x_test)
```

In [18]:
```
print("Accuracy :", accuracy_score(y_test, prediction))
print("\nClassification Report :")
print(classification_report(y_test, prediction))
print("\nConfusion Matrix :")
print(confusion_matrix(y_test, prediction))
```

```
Accuracy : 1.0

Classification Report :
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

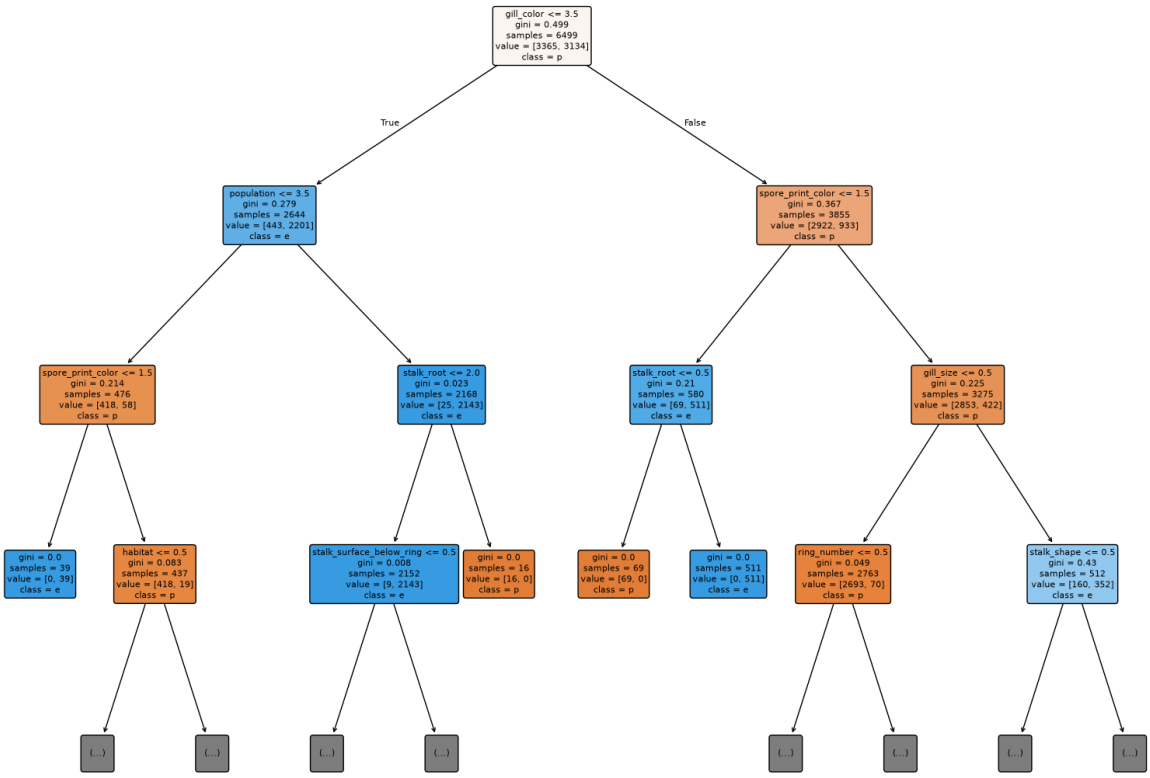
    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625


Confusion Matrix :
[[843   0]
 [  0 782]]
```

In [19]:

```python
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 15))
plot_tree(
    classifier,
    feature_names=x.columns,
    class_names=[str(c) for c in y.unique()],
    filled=True,
    rounded=True,
    fontsize=8,
    max_depth=3          # sirf 3 levels tak dikhao (warna bohot bara ho jayega)
)
plt.title("Decision Tree - Mushroom Classification", fontsize=16)
plt.show()
```

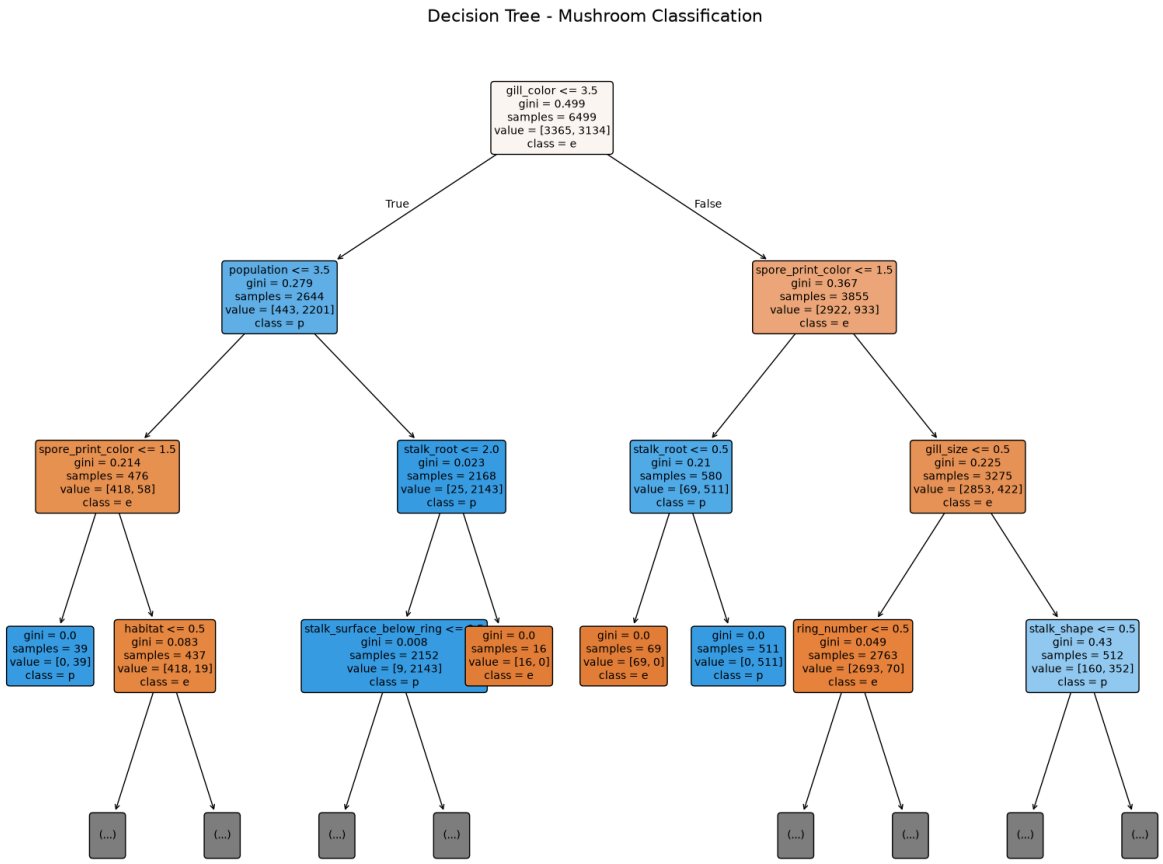


In [20]:
```
!pip install graphviz
```

```
Requirement already satisfied: graphviz in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (0.21)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [21]:
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 15))
plot_tree(
    classifier,
    feature_names=x.columns.tolist(),   # .tolist() zaroori hai
    class_names=['e', 'p'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=10
)
plt.title("Decision Tree - Mushroom Classification", fontsize=16)
plt.show()
```



In [22]:

```python
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Poora data encode karo
x_encoded = x.copy()
for col in x_encoded.columns:
    le = LabelEncoder()
    x_encoded[col] = le.fit_transform(x_encoded[col])

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

# Full Tree (overfitting check)
model_full = DecisionTreeClassifier(random_state=42)
scores_full = cross_val_score(model_full, x_encoded, y_encoded, cv=5)
print("=== Full Tree ===")
print("CV Scores:", scores_full)
print("Mean:", scores_full.mean())

# Limited Tree
model_limited = DecisionTreeClassifier(max_depth=5, random_state=42)
scores_limited = cross_val_score(model_limited, x_encoded, y_encoded, cv=5)
print("\n=== Limited Tree (max_depth=5) ===")
print("CV Scores:", scores_limited)
print("Mean:", scores_limited.mean())
```

```
=== Full Tree ===
CV Scores: [0.88184615 1.         1.         1.         0.92610837]
Mean: 0.9615909056460781

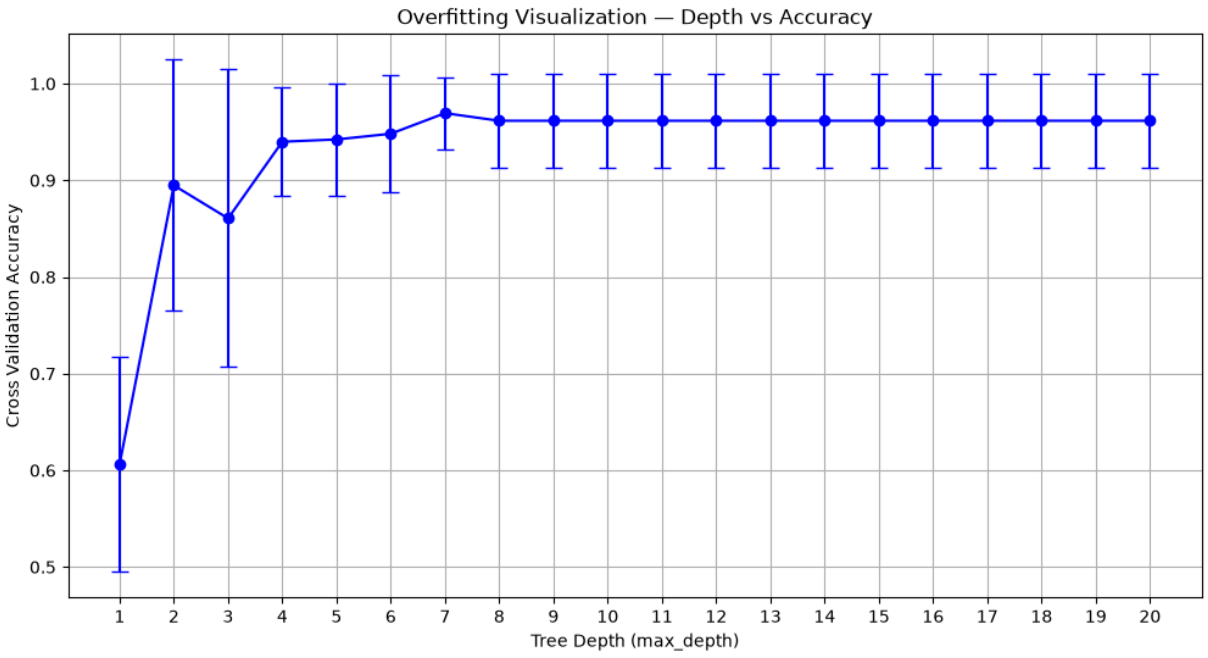
=== Limited Tree (max_depth=5) ===
CV Scores: [0.84246154 0.99692308 0.95076923 1.         0.92118227]
Mean: 0.9422672224327396
```

In [ ]:
```python
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

# Different depths try karo
depths = range(1, 21)
means = []
stds = []

for d in depths:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_val_score(model, x_encoded, y_encoded, cv=5)
    means.append(scores.mean())
    stds.append(scores.std())

# Plot
plt.figure(figsize=(12, 6))
plt.errorbar(depths, means, yerr=stds, marker='o', capsize=5, color='blue')
plt.xlabel("Tree Depth (max_depth)")
plt.ylabel("Cross Validation Accuracy")
plt.title("Overfitting Visualization — Depth vs Accuracy")
plt.grid(True)
plt.xticks(depths)
plt.show()
```



In [24]:
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
```

```
print("Depth | Accuracy")
print("-" * 25)

for d in [1, 2, 3, 5, 10, 15, 20, 25]:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_val_score(model, x_encoded, y_encoded, cv=5)
    print(f"  {d:2d}  |  {scores.mean():.4f}")
```

```
Depth | Accuracy
-------------------------
   1  |  0.6062
   2  |  0.8948
   3  |  0.8606
   5  |  0.9423
  10  |  0.9616
  15  |  0.9616
  20  |  0.9616
  25  |  0.9616
```

In [25]:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

print("Depth | Mean  | Std   | Reliable?")
print("-" * 40)

for d in [1, 2, 3, 5, 10, 15, 20]:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_val_score(model, x_encoded, y_encoded, cv=5)
    mean = scores.mean()
    std = scores.std()

    reliable = "✅ Haan" if std < 0.02 else "⚠️ Nahi"
    print(f"  {d:2d}  | {mean:.4f} | {std:.4f} | {reliable}")
```

```
Depth | Mean  | Std   | Reliable?
----------------------------------------
   1  | 0.6062 | 0.1115 | ⚠️ Nahi
   2  | 0.8948 | 0.1299 | ⚠️ Nahi
   3  | 0.8606 | 0.1540 | ⚠️ Nahi
   5  | 0.9423 | 0.0580 | ⚠️ Nahi
  10  | 0.9616 | 0.0491 | ⚠️ Nahi
  15  | 0.9616 | 0.0491 | ⚠️ Nahi
  20  | 0.9616 | 0.0491 | ⚠️ Nahi
```

In [26]:

```
from sklearn.preprocessing import LabelEncoder

# Poora data encode karo
x_encoded = x.copy()
for col in x_encoded.columns:
    le = LabelEncoder()
    x_encoded[col] = le.fit_transform(x_encoded[col])

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)
```

```
print("✅ Encoding done!")
print("x_encoded shape:", x_encoded.shape)
print("y_encoded shape:", y_encoded.shape)
```

```
✅ Encoding done!
x_encoded shape: (8124, 22)
y_encoded shape: (8124,)
```

In [27]:
```
from sklearn.preprocessing import LabelEncoder

# Poora data encode karo
x_encoded = x.copy()
for col in x_encoded.columns:
    le = LabelEncoder()
    x_encoded[col] = le.fit_transform(x_encoded[col])

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

print("✅ Encoding done!")
print("x_encoded shape:", x_encoded.shape)
print("y_encoded shape:", y_encoded.shape)
```

```
✅ Encoding done!
x_encoded shape: (8124, 22)
y_encoded shape: (8124,)
```

In [28]:
```
import pandas as pd
df = pd.read_csv("mushrooms.csv")
print(df.columns.tolist())
```

```
['type', 'cap_shape', 'cap_surface', 'cap_color', 'bruises', 'odor', 'gill_attachmen
t', 'gill_spacing', 'gill_size', 'gill_color', 'stalk_shape', 'stalk_root', 'stalk_s
urface_above_ring', 'stalk_surface_below_ring', 'stalk_color_above_ring', 'stalk_col
or_below_ring', 'veil_type', 'veil_color', 'ring_number', 'ring_type', 'spore_print_
color', 'population', 'habitat']
```

In [1]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Data load
df = pd.read_csv("mushrooms.csv")

# ⚠️ "type" use karo, "class" nahi
x = df.drop("type", axis=1)
y = df["type"]

# Encoding
x_encoded = x.copy()
for col in x_encoded.columns:
    le = LabelEncoder()
    x_encoded[col] = le.fit_transform(x_encoded[col])

le_y = LabelEncoder()
```

```
y_encoded = le_y.fit_transform(y)

# Table
print("Depth | Mean  | Std   | Reliable?")
print("-" * 40)

for d in [1, 2, 3, 5, 10, 15, 20]:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_val_score(model, x_encoded, y_encoded, cv=5)
    mean = scores.mean()
    std = scores.std()
    reliable = "✅ Haan" if std < 0.02 else "⚠️ Nahi"
    print(f"  {d:2d}  | {mean:.4f} | {std:.4f} | {reliable}")
```

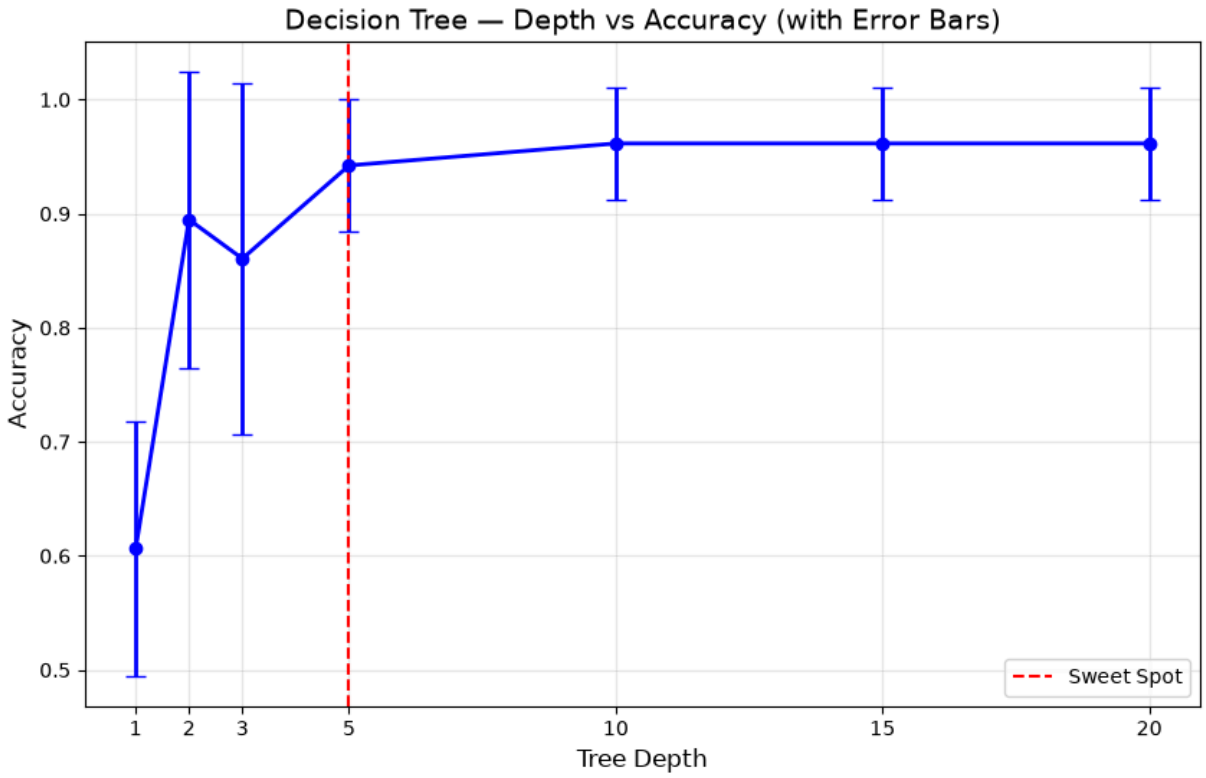
```
Depth | Mean  | Std   | Reliable?
----------------------------------------
   1  | 0.6062 | 0.1115 | ⚠️ Nahi
   2  | 0.8948 | 0.1299 | ⚠️ Nahi
   3  | 0.8606 | 0.1540 | ⚠️ Nahi
   5  | 0.9423 | 0.0580 | ⚠️ Nahi
  10  | 0.9616 | 0.0491 | ⚠️ Nahi
  15  | 0.9616 | 0.0491 | ⚠️ Nahi
  20  | 0.9616 | 0.0491 | ⚠️ Nahi
```

In [30]:

```
import matplotlib.pyplot as plt

depths = [1, 2, 3, 5, 10, 15, 20]
means = [0.6062, 0.8948, 0.8606, 0.9423, 0.9616, 0.9616, 0.9616]
stds = [0.1115, 0.1299, 0.1540, 0.0580, 0.0491, 0.0491, 0.0491]

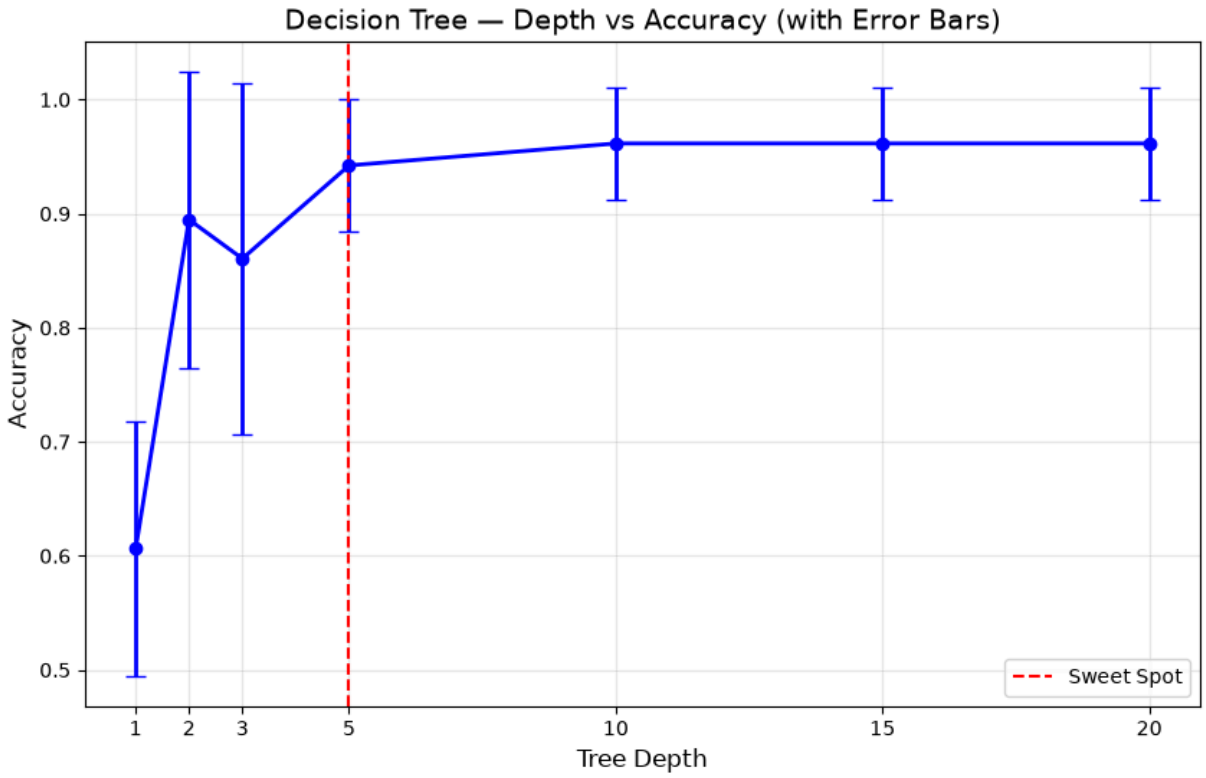
plt.figure(figsize=(10, 6))
plt.errorbar(depths, means, yerr=stds, marker='o', capsize=5, color='blue', linewid
plt.xlabel("Tree Depth", fontsize=12)
plt.ylabel("Accuracy", fontsize=12)
plt.title("Decision Tree — Depth vs Accuracy (with Error Bars)", fontsize=14)
plt.grid(True, alpha=0.3)
plt.xticks(depths)
plt.axvline(x=5, color='red', linestyle='--', label='Sweet Spot')
plt.legend()
plt.show()
```



In [31]:
```
import matplotlib.pyplot as plt

depths = [1, 2, 3, 5, 10, 15, 20]
means = [0.6062, 0.8948, 0.8606, 0.9423, 0.9616, 0.9616, 0.9616]
stds = [0.1115, 0.1299, 0.1540, 0.0580, 0.0491, 0.0491, 0.0491]

plt.figure(figsize=(10, 6))
plt.errorbar(depths, means, yerr=stds, marker='o', capsize=5, color='blue', linewid
plt.xlabel("Tree Depth", fontsize=12)
plt.ylabel("Accuracy", fontsize=12)
plt.title("Decision Tree — Depth vs Accuracy (with Error Bars)", fontsize=14)
plt.grid(True, alpha=0.3)
plt.xticks(depths)
plt.axvline(x=5, color='red', linestyle='--', label='Sweet Spot')
plt.legend()
plt.show()
```



In [ ]:

In [32]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Hyperparameters ki list
param_grid = {
    'max_depth': [1, 2, 3, 5, 7, 10, 15, 20],
    'criterion': ['gini', 'entropy'],
    'min_samples_split': [2, 5, 10]
}

# GridSearchCV
grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# Train
grid.fit(x_encoded, y_encoded)

# Best
print("=" * 50)
print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
print("=" * 50)
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
==================================================
Best Parameters: {'criterion': 'gini', 'max_depth': 7, 'min_samples_split': 2}
Best Score: 0.9694678287230012
==================================================
```

In [33]:

```python
# ================================
# 1. IMPORTS
# ================================
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder

# ================================
# 2. DATA LOAD
# ================================
df = pd.read_csv("mushrooms.csv")
x = df.drop("type", axis=1)      # ⚠️ "type" likho, "class" nahi
y = df["type"]

# ================================
# 3. ENCODING
# ================================
x_encoded = x.copy()
for col in x_encoded.columns:
    le = LabelEncoder()
    x_encoded[col] = le.fit_transform(x_encoded[col])

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

print("✅ Encoding done! Shape:", x_encoded.shape)

# ================================
# 4. GRID SEARCH
# ================================
param_grid = {
    'max_depth': [1, 2, 3, 5, 7, 10, 15, 20],
    'criterion': ['gini', 'entropy'],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid.fit(x_encoded, y_encoded)

# ================================
# 5. RESULT
```

```
# ================================
print("=" * 50)
print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
print("=" * 50)
```

```
✅ Encoding done! Shape: (8124, 22)
Fitting 5 folds for each of 48 candidates, totalling 240 fits
==================================================
Best Parameters: {'criterion': 'gini', 'max_depth': 7, 'min_samples_split': 2}
Best Score: 0.9694678287230012
==================================================
```

In [34]:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Best model nikalo
best_model = grid.best_estimator_

# Train/Test split
x_train, x_test, y_train, y_test = train_test_split(
    x_encoded, y_encoded, test_size=0.2, random_state=42
)

# Best model train karo
best_model.fit(x_train, y_train)

# Prediction
y_pred = best_model.predict(x_test)

# Accuracy
train_acc = best_model.score(x_train, y_train)
test_acc = best_model.score(x_test, y_test)

print("=" * 50)
print(f"Train Accuracy: {train_acc:.4f}")
print(f"Test Accuracy:  {test_acc:.4f}")
print(f"Difference:     {train_acc - test_acc:.4f}")
print("=" * 50)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
==================================================
Train Accuracy: 1.0000
Test Accuracy:  1.0000
Difference:     0.0000
==================================================

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625


Confusion Matrix:
[[843   0]
 [  0 782]]
```

In [35]:
```python
import pandas as pd

# Saare results dekho
results = pd.DataFrame(grid.cv_results_)
results = results[['param_max_depth', 'param_criterion', 'param_min_samples_split',
results = results.sort_values('mean_test_score', ascending=False)

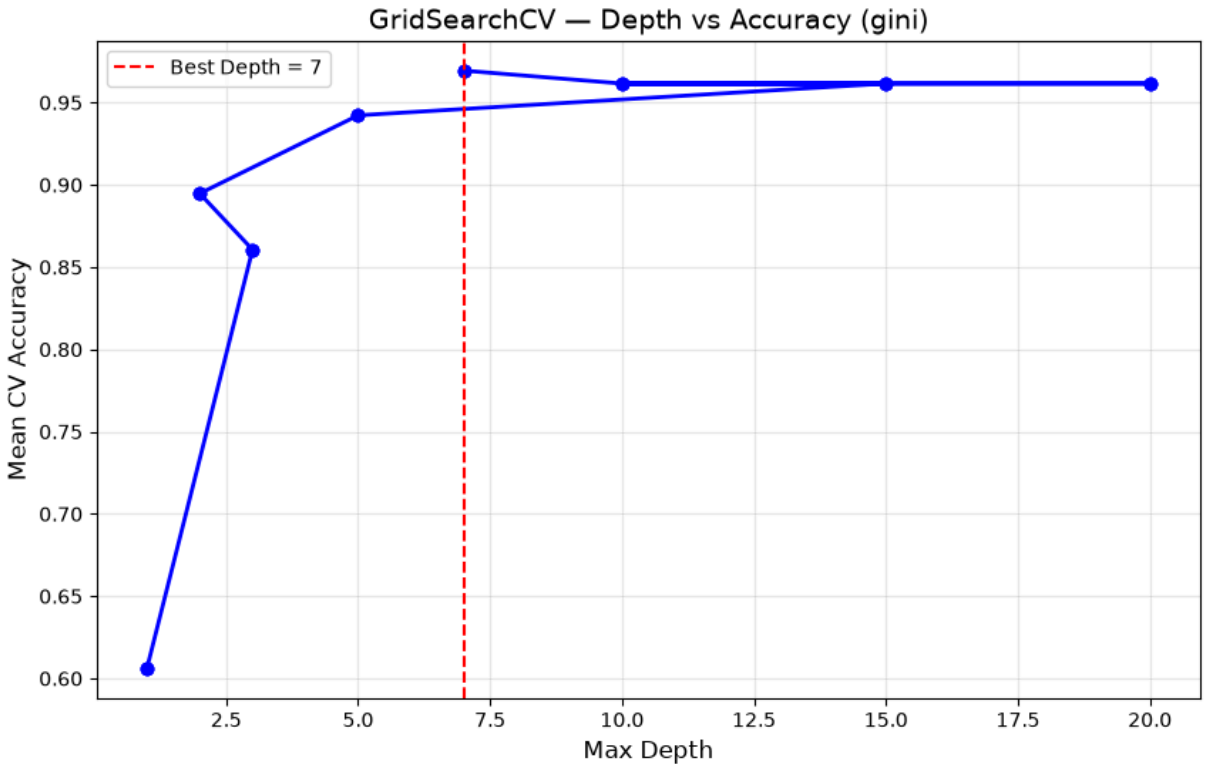
print("Top 10 Best Combinations:")
print(results.head(10).to_string(index=False))
```

```
Top 10 Best Combinations:
 param_max_depth param_criterion  param_min_samples_split  mean_test_score  std_test
_score
               7            gini                       10         0.969468        0.
037426
               7            gini                        2         0.969468        0.
037426
               7            gini                        5         0.969468        0.
037426
              10            gini                        2         0.961591        0.
049080
              15            gini                        5         0.961591        0.
049080
              15            gini                        2         0.961591        0.
049080
              10            gini                       10         0.961591        0.
049080
              10            gini                        5         0.961591        0.
049080
              20            gini                       10         0.961591        0.
049080
              20            gini                        5         0.961591        0.
049080
```

In [36]:
```
import matplotlib.pyplot as plt

# Depth vs Accuracy (best criterion ke saath)
results_gini = results[results['param_criterion'] == 'gini']

plt.figure(figsize=(10, 6))
plt.plot(results_gini['param_max_depth'], results_gini['mean_test_score'],
         marker='o', linewidth=2, color='blue')
plt.xlabel("Max Depth", fontsize=12)
plt.ylabel("Mean CV Accuracy", fontsize=12)
plt.title("GridSearchCV — Depth vs Accuracy (gini)", fontsize=14)
plt.grid(True, alpha=0.3)
plt.axvline(x=7, color='red', linestyle='--', label='Best Depth = 7')
plt.legend()
plt.show()
```



In [37]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# Random Forest (100 trees)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
scores = cross_val_score(rf, x_encoded, y_encoded, cv=5)

print("Random Forest CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
Random Forest CV Scores: [0.84246154 1.         1.         1.         0.58435961]
Mean: 0.8853642288745738
Std: 0.1623998177058611
```

In [38]:
```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# Data load
df = pd.read_csv("mushrooms.csv")
x = df.drop("type", axis=1)
y = df["type"]

# OneHotEncoder
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
x_onehot = encoder.fit_transform(x)

# Target encode
from sklearn.preprocessing import LabelEncoder
le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

print("Original shape:", x.shape)
print("OneHot shape:", x_onehot.shape)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
scores = cross_val_score(rf, x_onehot, y_encoded, cv=5)

print("\n=== Random Forest (OneHot) ===")
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
Original shape: (8124, 22)
OneHot shape: (8124, 117)

=== Random Forest (OneHot) ===
CV Scores: [0.84246154 1.         0.99938462 1.         0.8158867 ]
Mean: 0.9315465706707086
Std: 0.08400844367765795
```

In [39]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rf = RandomForestClassifier(
    n_estimators=200,        # 100 se 200 karo
    max_depth=10,            # Trees ko limit karo
    random_state=42,
    n_jobs=-1
)

scores = cross_val_score(rf, x_onehot, y_encoded, cv=5)
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
CV Scores: [0.84246154 1.         1.         1.         0.79802956]
Mean: 0.928098219022357
Std: 0.08917521663782671
```

In [40]:

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Best model nikalo
best_model = grid.best_estimator_

# Train/Test split
x_train, x_test, y_train, y_test = train_test_split(
    x_encoded, y_encoded, test_size=0.2, random_state=42
)

# Best model train karo
best_model.fit(x_train, y_train)

# Prediction
y_pred = best_model.predict(x_test)

# Accuracy
train_acc = best_model.score(x_train, y_train)
test_acc = best_model.score(x_test, y_test)

print("=" * 50)
print(f"Train Accuracy: {train_acc:.4f}")
print(f"Test Accuracy:  {test_acc:.4f}")
print(f"Difference:     {train_acc - test_acc:.4f}")
print("=" * 50)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
==================================================
Train Accuracy: 1.0000
Test Accuracy:  1.0000
Difference:     0.0000
==================================================

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       843
           1       1.00      1.00      1.00       782

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625


Confusion Matrix:
[[843   0]
 [  0 782]]
```

In [41]:
```python
import pandas as pd

# Sab models ka result
comparison = pd.DataFrame({
    'Model': [
        'Decision Tree (LabelEnc)',
        'Decision Tree (Tuned)',
        'Random Forest (LabelEnc)',
        'Random Forest (OneHot)',
        'Random Forest (Tuned+Strat)'
    ],
    'Mean CV Accuracy': [
        0.9423,
        0.9695,
        0.8853,
        0.9315,
        None  # jab chalao to bharo
    ],
    'Std': [
        0.0580,
        0.0374,
        0.1624,
        0.0840,
        None
    ]
})

print(comparison.to_string(index=False))
```

```
                     Model  Mean CV Accuracy    Std
   Decision Tree (LabelEnc)            0.9423 0.0580
      Decision Tree (Tuned)            0.9695 0.0374
   Random Forest (LabelEnc)            0.8853 0.1624
     Random Forest (OneHot)            0.9315 0.0840
Random Forest (Tuned+Strat)               NaN    NaN
```

In [42]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
import pandas as pd

# Data Load
df = pd.read_csv("mushrooms.csv")
x = df.drop("type", axis=1)
y = df["type"]

# OneHot Encoding
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
x_onehot = encoder.fit_transform(x)

le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

# Stratified K-Fold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Tuned Random Forest
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    min_samples_split=2,
    random_state=42,
    n_jobs=-1
)

scores = cross_val_score(rf, x_onehot, y_encoded, cv=skf)
print("=== Random Forest (Tuned + OneHot + Stratified) ===")
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
=== Random Forest (Tuned + OneHot + Stratified) ===
CV Scores: [1. 1. 1. 1. 1.]
Mean: 1.0
Std: 0.0
```

In [43]:
```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (2.4.6)
Requirement already satisfied: scipy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (1.17.1)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [44]:
```
from xgboost import XGBClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
xgb = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    random_state=42,
    eval_metric='logloss'
)

scores = cross_val_score(xgb, x_onehot, y_encoded, cv=skf)
print("=== XGBoost ===")
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
=== XGBoost ===
CV Scores: [1. 1. 1. 1. 1.]
Mean: 1.0
Std: 0.0
```

In [45]:
```
import xgboost
print("XGBoost version:", xgboost.__version__)
```

```
XGBoost version: 3.2.0
```

In [46]:
```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (2.4.6)
Requirement already satisfied: scipy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (1.17.1)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [47]:
```
import xgboost
print("XGBoost version:", xgboost.__version__)
```

```
XGBoost version: 3.2.0
```

In [48]:
```
import sys
print(sys.executable)
```

```
c:\Users\HP\Desktop\mashrooms .csv\ml_env\Scripts\python.exe
```

In [49]:
```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (2.4.6)
Requirement already satisfied: scipy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (from xgboost) (1.17.1)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [50]:
```
import xgboost
print("XGBoost version:", xgboost.__version__)
```

```
XGBoost version: 3.2.0
```

In [51]:
```
import sys
print(sys.executable)
```

```
c:\Users\HP\Desktop\mashrooms .csv\ml_env\Scripts\python.exe
```

In [52]:
```
import xgboost
print("XGBoost version:", xgboost.__version__)
```

```
XGBoost version: 3.2.0
```

In [53]:
```
from xgboost import XGBClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

xgb = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    random_state=42,
    eval_metric='logloss'
)

scores = cross_val_score(xgb, x_onehot, y_encoded, cv=skf)

print("=== XGBoost ===")
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
=== XGBoost ===
CV Scores: [1. 1. 1. 1. 1.]
Mean: 1.0
Std: 0.0
```

In [54]:
```
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in c:\users\hp\desktop\mashrooms .csv\ml
_env\lib\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in c:\users\hp\desktop\mashrooms .csv\m
l_env\lib\site-packages (from scikit-learn) (2.4.6)
Requirement already satisfied: scipy>=1.10.0 in c:\users\hp\desktop\mashrooms .csv\m
l_env\lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.4.0 in c:\users\hp\desktop\mashrooms .csv\m
l_env\lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in c:\users\hp\desktop\mashrooms .csv
\ml_env\lib\site-packages (from scikit-learn) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in c:\users\hp\desktop\mashrooms
.csv\ml_env\lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in c:\users\hp\desktop\mashrooms .cs
v\ml_env\lib\site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [55]:
```
!pip install scikit-learn pandas numpy matplotlib seaborn
```

```
Requirement already satisfied: scikit-learn in c:\users\hp\desktop\mashrooms .csv\ml
_env\lib\site-packages (1.9.1)
Requirement already satisfied: pandas in c:\users\hp\desktop\mashrooms .csv\ml_env\l
ib\site-packages (3.0.6)
Requirement already satisfied: numpy in c:\users\hp\desktop\mashrooms .csv\ml_env\li
b\site-packages (2.4.6)
Requirement already satisfied: matplotlib in c:\users\hp\desktop\mashrooms .csv\ml_e
nv\lib\site-packages (3.11.2)
Requirement already satisfied: seaborn in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (0.13.2)
Requirement already satisfied: scipy>=1.10.0 in c:\users\hp\desktop\mashrooms .csv\m
l_env\lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.4.0 in c:\users\hp\desktop\mashrooms .csv\m
l_env\lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in c:\users\hp\desktop\mashrooms .csv
\ml_env\lib\site-packages (from scikit-learn) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in c:\users\hp\desktop\mashrooms
.csv\ml_env\lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\desktop\mashroo
ms .csv\ml_env\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\hp\desktop\mashrooms .csv\ml_env\l
ib\site-packages (from pandas) (2026.5)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\desktop\mashrooms .cs
v\ml_env\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\desktop\mashrooms .csv\ml
_env\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\users\hp\desktop\mashrooms .c
sv\ml_env\lib\site-packages (from matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\desktop\mashrooms .c
sv\ml_env\lib\site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\desktop\mashrooms .csv
\ml_env\lib\site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in c:\users\hp\desktop\mashrooms .csv\ml_en
v\lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in c:\users\hp\desktop\mashrooms .csv\ml
_env\lib\site-packages (from matplotlib) (3.3.3)
Requirement already satisfied: cloudpickle>=3.0 in c:\users\hp\desktop\mashrooms .cs
v\ml_env\lib\site-packages (from joblib>=1.4.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in c:\users\hp\desktop\mashrooms .csv\ml_env
\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [56]:

```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.model_selection import cross_val_score, StratifiedKFold
from xgboost import XGBClassifier

# Data load
df = pd.read_csv("mushrooms.csv")
x = df.drop("type", axis=1)
y = df["type"]

# OneHot Encoding
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
x_onehot = encoder.fit_transform(x)
```

```
le_y = LabelEncoder()
y_encoded = le_y.fit_transform(y)

print("Ready! Shape:", x_onehot.shape)

# XGBoost
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

xgb = XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    random_state=42,
    eval_metric='logloss'
)

scores = cross_val_score(xgb, x_onehot, y_encoded, cv=skf)

print("\n=== XGBoost ===")
print("CV Scores:", scores)
print("Mean:", scores.mean())
print("Std:", scores.std())
```

```
Ready! Shape: (8124, 117)

=== XGBoost ===
CV Scores: [1. 1. 1. 1. 1.]
Mean: 1.0
Std: 0.0
```

In [57]:

```
import matplotlib.pyplot as plt
import numpy as np

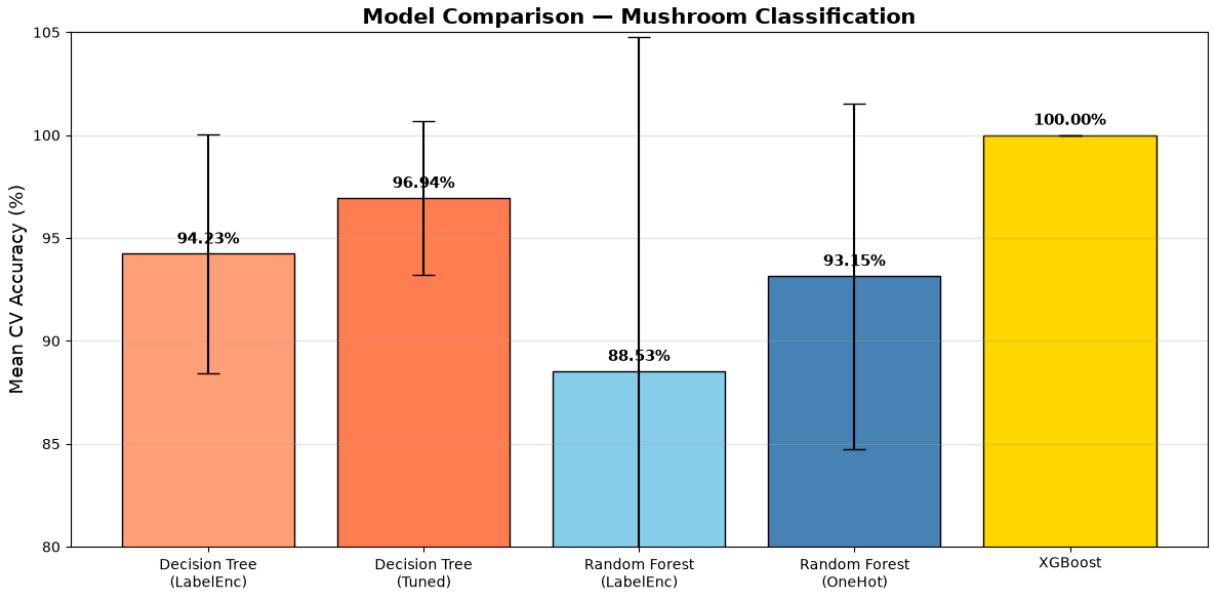
# Data
models = [
    'Decision Tree\n(LabelEnc)',
    'Decision Tree\n(Tuned)',
    'Random Forest\n(LabelEnc)',
    'Random Forest\n(OneHot)',
    'XGBoost'
]
means = [94.23, 96.94, 88.53, 93.15, 100.0]
stds = [5.80, 3.74, 16.24, 8.40, 0.0]

# Colors
colors = ['#FFA07A', '#FF7F50', '#87CEEB', '#4682B4', '#FFD700']

# Graph
plt.figure(figsize=(12, 6))
bars = plt.bar(models, means, yerr=stds, capsize=8, color=colors, edgecolor='black'

# Values upar likho
for bar, mean in zip(bars, means):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
             f'{mean:.2f}%', ha='center', fontsize=11, fontweight='bold')
```

```
plt.ylabel("Mean CV Accuracy (%)", fontsize=13)
plt.title("Model Comparison — Mushroom Classification", fontsize=15, fontweight='bo
plt.ylim(80, 105)
plt.grid(True, alpha=0.3, axis='y')
plt.xticks(fontsize=10)
plt.tight_layout()
plt.show()
```



In [58]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Data load karo
df = pd.read_csv("mushrooms.csv")

# 1. Data ka size dekho
print("Total Rows aur Columns:", df.shape)

# 2. Data ke types dekho (kya koi number hai ya sab text hai?)
print("\nData Types:\n", df.dtypes.head())

# 3. Missing values check karo
print("\nMissing Values:", df.isnull().sum().sum())
```

```
Total Rows aur Columns: (8124, 23)

Data Types:
 type           str
cap_shape      str
cap_surface    str
cap_color      str
bruises        str
dtype: object

Missing Values: 0
```

In [59]:
```
# Target column ('type') ki counting
print("Mushroom Types:\n", df["type"].value_counts())

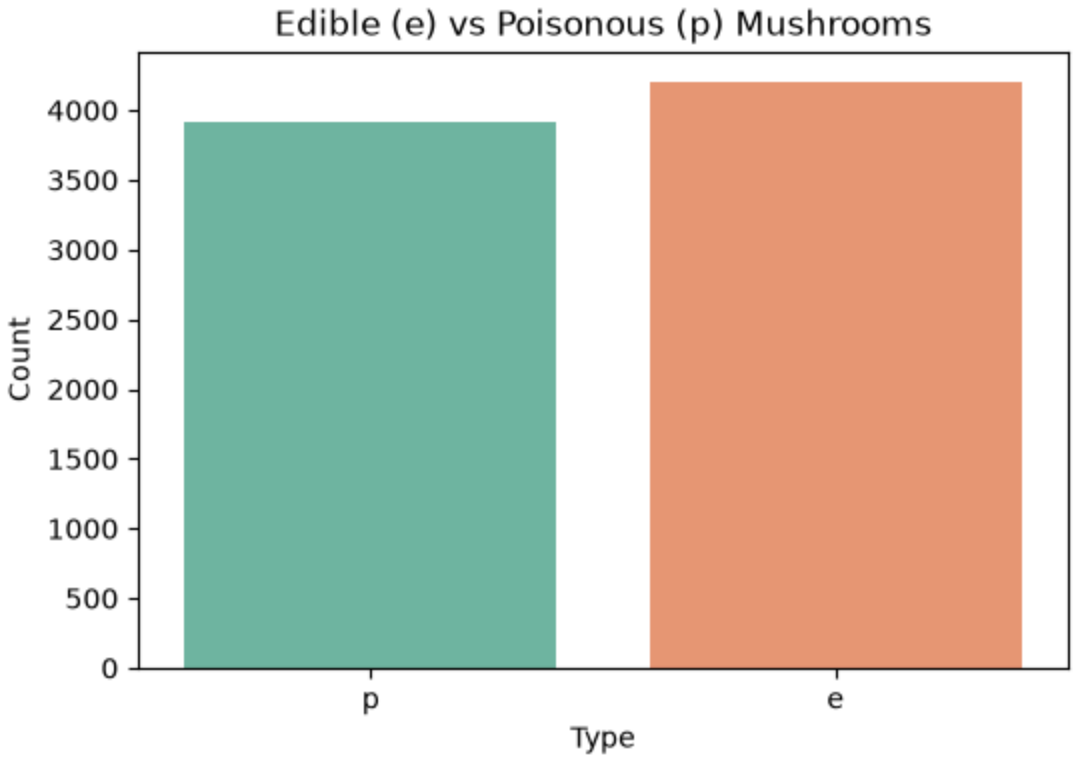
# Isko graph mein dekhte hain
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='type', palette='Set2')
plt.title("Edible (e) vs Poisonous (p) Mushrooms")
plt.xlabel("Type")
plt.ylabel("Count")
plt.show()
```

```
Mushroom Types:
 type
e    4208
p    3916
Name: count, dtype: int64
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_17128\3071193189.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.1
4.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(data=df, x='type', palette='Set2')
```

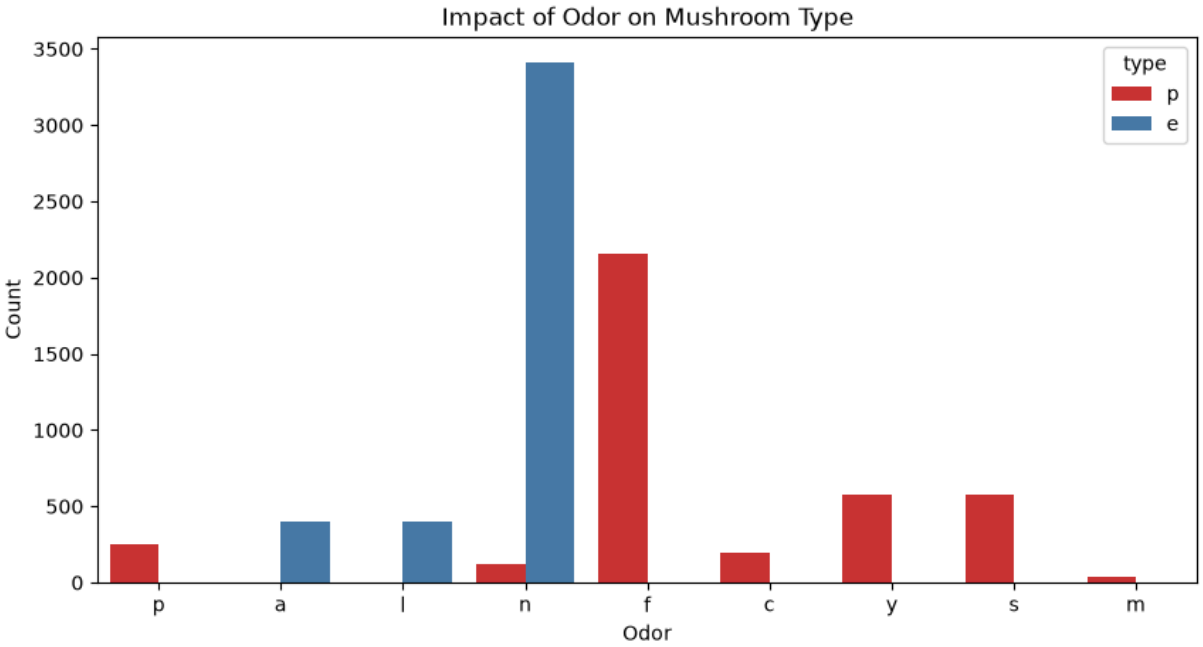


In [60]:
```
# Odor aur Type ka relation dekho
print("Odor vs Type:\n", pd.crosstab(df['odor'], df['type']))

# Isko graph mein visualize karo
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x='odor', hue='type', palette='Set1')
plt.title("Impact of Odor on Mushroom Type")
plt.xlabel("Odor")
```

```
plt.ylabel("Count")
plt.show()
```

```
Odor vs Type:
 type     e     p
odor
a      400     0
c        0   192
f        0  2160
l      400     0
m        0    36
n     3408   120
p        0   256
s        0   576
y        0   576
```



In [61]:
```
# Ek chhoti summary table banao
summary = df.groupby('type').mean(numeric_only=True) # (Yeh sirf numbers par kaam k

# Ek naya Excel file banayein jisme data ho
# df.to_excel("Mushroom_Data_Report.xlsx", index=False)
# print("Report saved successfully!")

# Abhi ke liye hum sirf ek table display karte hain
print("Report Summary Generated!")
```

```
Report Summary Generated!
```

In [63]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Graph banao
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x='odor', hue='type', palette='Set1')
plt.title("Impact of Odor on Mushroom Type")

# Graph ko save karo
```

```
plt.savefig("odor_graph.png", dpi=150, bbox_inches='tight')
plt.show()
```

